

Вступ до Helm

Що таке Helm?

Helm — це потужний менеджер пакетів для Kubernetes, який спрощує розгортання, масштабування та управління додатками в середовищі Kubernetes. Подібно до менеджерів пакетів у Linux (таких як APT або Yum), Helm допомагає автоматизувати розгортання складних програм, організовуючи ресурси Kubernetes у багаторазові, спільні одиниці, які називаються «діаграмами». За допомогою Helm користувачі можуть ефективніше та послідовніше розгортати, оновлювати та відкочувати програми та служби на кластерах Kubernetes. Helm Charts дозволяють розробникам об'єднувати повний набір конфігурацій та залежностей Kubernetes, спрощуючи розгортання навіть найскладніших програм.

Потреба в Helm

Kubernetes, хоча й потужний, вимагає ручного керування кількома взаємопов'язаними YAML-файлами для кожного ресурсу, такого як розгортання, служби та правила вхідного процесу. Оскільки додатки стають складнішими та базуються на мікросервісах, керування цими конфігураціями стає складним, особливо коли йдеться про керування версіями, шаблони та обробку залежностей. Helm було розроблено для вирішення цієї складності, забезпечуючи структурований спосіб управління та розгортання колекцій ресурсів Kubernetes. Завдяки впровадженню Helm, користувачі Kubernetes отримують можливість автоматизувати процес керування кількома ресурсами, забезпечуючи узгодженість, легкість розгортання та можливість повторного використання в різних середовищах. Helm ефективно оптимізує управління життєвим циклом додатків у Kubernetes.

Переваги використання Helm у Kubernetes

Helm пропонує численні переваги в середовищі Kubernetes, головним чином завдяки підвищенню ефективності, узгодженості та контролю над розгортаннями:

- 1. Спрощене розгортання:** Helm-чарти об'єднують усі необхідні ресурси Kubernetes, що спрощує розгортання програм за допомогою однієї команди.
- 2. Контроль версій та відкати:** Helm дозволяє керувати різними версіями ваших програм, що спрощує відкат до попередньої версії за потреби.
- 3. Налаштування та повторне використання:** Діаграми Helm можна налаштовувати за допомогою файлів значень, що робить конфігурації гнучкими та повторно використовуваними для різних середовищ.
- 4. Керування залежностями:** Helm допомагає керувати залежностями між сервісами або програмами, забезпечуючи їх розгортання в правильному порядку та з необхідними конфігураціями.
- 5. Спільнота та екосистема:** Helm має багату екосистему з різноманітними готовими діаграмами, що пришвидшує розгортання поширених програм.

Як Helm вписується в DevOps

Helm ідеально відповідає принципам DevOps, забезпечуючи ефективні, автоматизовані та узгоджені робочі процеси розгортання в Kubernetes. У DevOps метою є оптимізація та автоматизація процесу розгортання для забезпечення швидких та надійних релізів. Helm покращує це, надаючи версії шаблонів визначень програм, які можна легко розгортати, керувати та оновлювати в різних середовищах. За допомогою Helm команди DevOps можуть автоматизувати розгортання та оновлення складних додатків, контролювати релізи за допомогою відкатів та розгортати інфраструктуру як код (IaC), що відповідає практикам безперервної інтеграції та безперервної доставки (CI/CD). Коротше кажучи, Helm виступає мостом між розробкою та експлуатацією, надаючи командам можливість швидше та надійніше створювати додатки в екосистемі Kubernetes.

Порівняння з традиційними методами розгортання

Традиційні методи розгортання зазвичай передбачають розгортання програм безпосередньо на віртуальних машинах або серверах, ручне керування конфігураціями та часто стикаються з проблемами масштабування та узгодженості. Цим методам бракує оркестрації, гнучкості та опцій відкату, які є в контейнерних середовищах. На відміну від цього, Helm, у поєднанні з Kubernetes, пропонує декларативне, версійне та автоматизоване розгортання, де конфігурації інфраструктури та застосунків упаковані разом у діаграми. Це забезпечує швидше розгортання, простіше масштабування та легке відкату порівняно з традиційними методами. Крім того, система шаблонів та значень Helm замінює ручне налаштування конфігурації, зменшуючи людський фактор та підвищуючи узгодженість у різних середовищах.

Версії Helm та ключові зміни

Helm значно еволюціонував з моменту свого першого випуску, і кожна основна версія приносила покращення, нові функції та структурні зміни для покращення зручності використання, безпеки та продуктивності. Основні версії Helm включають:

- **Helm 2:** Введено концепцію Helm Charts та використано компонент під назвою Tiller для керування розгортаннями. Однак, присутність Тіллера викликала занепокоєння щодо безпеки, оскільки вимагала підвищених дозволів у кластері Kubernetes.
- **Helm 3:** Видалено кермо, що значно покращує безпеку, усуваючи необхідність у дозволах для всього кластера. У Helm 3 було запроваджено більш спрощену архітектуру, покращене керування діаграмами та підтримку CRD (визначень користувацьких ресурсів) як частину встановлення діаграм. Крім того, Helm 3 запропонував покращену підтримку сучасних функцій Kubernetes, таких як контроль доступу на основі ролей (RBAC) та простори імен.

Архітектура Helm

- **Клієнт Helm:** Клієнт Helm — це інструмент інтерфейсу командного рядка (CLI), який використовується розробниками та операторами. Він надсилає запити до API Kubernetes для встановлення, оновлення, видалення або запиту діаграм Helm та їхніх ресурсів. Клієнт обробляє взаємодію з репозиторіями, завантажує діаграми та керує життєвим циклом релізу.
- **Helm Charts:** Helm Chart — це попередньо налаштований пакет, який об'єднує файли ресурсів Kubernetes YAML, шаблони та значення, необхідні для розгортання програми. Діаграми містять налаштування за замовчуванням, але їх можна налаштувати відповідно до конкретних потреб розгортання.
- **Кластер Kubernetes:** Кластер Kubernetes розміщує та керує фактичними ресурсами, розгорнутими Helm, такими як pod-системи, сервіси та розгортання. Helm взаємодіє з API Kubernetes для виконання цих розгортань.
- **Керування релізами:** Коли діаграму Helm встановлено, вона відстежується як «реліз», що представляє собою екземпляр програми. Helm керує цими релізами, дозволяючи вам відстежувати, оновлювати або скасовувати версії вашої програми.

Налаштування Helm

Налаштування Helm включає встановлення Helm CLI та його налаштування для взаємодії з кластером Kubernetes.

Процес зазвичай включає:

- 1. Встановлення інтерфейсу командного рядка Helm:** Helm можна встановити на різні платформи за допомогою менеджерів пакетів, таких як brew для macOS, apt для Linux на базі Debian, або безпосередньо завантажити для інших систем. Після встановлення команда helm стає доступною в терміналі.
- 2. Налаштуйте доступ до Kubernetes:** Helm потребує доступу до кластера Kubernetes. Переконайтеся, що kubectl налаштовано та автентифіковано для підключення до вашого кластера Kubernetes, оскільки Helm покладається на конфігурації kubectl для доступу та розгортання ресурсів у кластері.
- 3. Ініціалізація Helm:** У Helm 3 ініціалізація є простою, оскільки Tiller (який використовується в Helm 2) більше не потрібен. Просто перевірте встановлення, запустивши команду helm version, яка підтверджує підключення Helm до кластера.

Розуміння репозиторіїв Helm

Репозиторії Helm є важливими для функціональності Helm, оскільки вони зберігають та розповсюджують діаграми Helm. Репозиторій — це колекція попередньо упакованих діаграм застосунків, які можна шукати, завантажувати та розгортати. Ключові аспекти включають:

- 1. Додавання репозиторію:** Команда `helm repo add` дозволяє користувачам додавати публічні або приватні репозиторії, такі як популярний Artifact Hub, який містить широкий спектр діаграм Helm. Наприклад, команда `helm repo add stable https://charts.helm.sh/stable` додає стандартний репозиторій Helm stable.
- 2. Пошук та оновлення:** Після додавання користувачі можуть шукати діаграми в репозиторії за допомогою `helm search` у сховищі <назва-діаграми>. Щоб репозиторії оновлювалися найновішими версіями діаграм, використовуйте оновлення репозиторіїв `helm`.
- 3. Користувацькі та приватні репозиторії:** Організації можуть створювати приватні репозиторії для внутрішніх програм, що додає безпеки та контролю над розгортанням.



Helm Charts

Що таке Helm Charts?

Діаграми Helm — це основний формат упаковки в Helm, що представляє собою колекцію YAML-файлів, які визначають, налаштовують та керують програмами або сервісами в Kubernetes. Діаграма Helm — це, по суті, попередньо налаштований додаток Kubernetes, що містить усі ресурси, необхідні для розгортання певного додатку або сервісу, такі як розгортання, сервіси та карти конфігурації. Діаграми спрощують об'єднання складних програм, дозволяючи користувачам розгорнути їх з мінімальними зусиллями, налаштовувати конфігурації за потреби та керувати залежностями. Діаграми Helm також дозволяють повторне використання, що дає змогу обмінюватися конфігураціями між командами або середовищами. Завдяки діаграмам Helm навіть складні програми можна розгорнути, оновлювати та підтримувати в Kubernetes за допомогою єдиного узгодженого пакета.

Компоненти діаграми

- **Chart.yaml:** Цей файл є основним дескриптором діаграми Helm, що містить метадані, такі як назва діаграми, версія, опис та залежності.
- **Values.yaml:** Файл значень за замовчуванням, де встановлюються налаштовувані параметри для діаграми. Користувачі можуть змінювати ці значення, щоб налаштувати розгортання без зміни основних файлів діаграм.
- **Каталог шаблонів:** Цей каталог містить шаблони YAML для ресурсів Kubernetes (наприклад, розгортань, сервісів) із заповнювачами, які Helm заповнює на основі значень з Values.yaml. Шаблони забезпечують гнучкість та можливість повторного використання в різних середовищах.
- **Каталог діаграм (необов'язково):** Цей каталог може містити залежності у вигляді інших діаграм, що дозволяє керувати та розгортати вкладені або залежні служби.
- **README.md (необов'язково):** Часто додається для надання документації, що пояснює, як ефективно використовувати та налаштовувати діаграму.

Chart.yaml

Chart.yaml – це ключовий файл у діаграмах Helm, який слугує дескриптором метаданих діаграми. Він визначає інформацію про саму діаграму та надає деталі, необхідні Helm для ефективного керування діаграмою. Ключові поля в Chart.yaml включають:

1. **назва:** Назва діаграми, яка часто відображає програму або послугу, яку вона представляє.
2. **версія:** Версія діаграми, що використовується для відстеження та керування версіями в Helm.
3. **опис:** короткий опис призначення діаграми або застосування, яке вона розгортає.
4. **appVersion:** Версія розгортаної програми (відрізняється від версії діаграми), що дозволяє користувачам відстежувати історію версій програми.
5. **залежності:** Перелічує інші діаграми, від яких залежить ця діаграма, які можуть бути розташовані в каталозі charts/ або витягнуті з репозиторію.
6. **ключові слова та супроводжуючі (необов'язково):** Використовуються для пошуку та атрибуції, що полегшує пошук або ідентифікацію діаграми.

```
apiVersion: v2
name: mychart
version: 1.0.0
description: "helm chart"
keywords:
  - mykeyword
home: http://example.com/
sources:
  - https://github.com/example/mychart
dependencies:
  - name: nginx
    version: "1.14.0"
    repository: "https://charts.helm.sh/stable"
```

Значення.yaml

- **Конфігурація за замовчуванням:** Values.yaml надає базовий набір значень конфігурації, таких як кількість реплік, теги зображень та обмеження ресурсів. Ці значення слугують параметрами за замовчуванням для розгортання Helm-діаграми.
- **Перевизначення значень:** Користувачі можуть створювати власні файли значень або передавати окремі значення під час встановлення чи оновлення (прапорець --set), щоб перевизначити значення за замовчуванням у Values.yaml. Така гнучкість дозволяє адаптувати розгортання до унікальних вимог або конкретних середовищ.
- **Ієрархічна структура:** Файл підтримує вкладені значення, що дозволяє створювати організовані, складні конфігурації, особливо для програм з кількома компонентами.

```
replicaCount: 2
image:
  repository: nginx
  tag: stable
  pullPolicy: IfNotPresent
service:
  type: NodePort
  port: 80
```

Файли шаблонів

Файли шаблонів у діаграмі Helm, що знаходяться в каталозі templates/, – це YAML-файли, які визначають ресурси Kubernetes (такі як Deployments, Services та ConfigMaps), необхідні для розгортання програми.

Ці файли написані з використанням синтаксису шаблонів Go, що дозволяє динамічну вставку значень з Values.yaml та умовну логіку. Ключові елементи файлів шаблонів включають:

- **Динамічні визначення ресурсів:** Файли шаблонів використовують заповнювачі (наприклад, {{ .Values.key }}) для вставки значень з Values.yaml, що забезпечує гнучкість та налаштування у згенерованих маніфестах Kubernetes.
- **Умовна логіка:** шаблони Helm можуть містити логічні оператори (наприклад, if, else) для динамічного налаштування ресурсів на основі конфігурації. Ця можливість корисна для додаткових компонентів або налаштувань, які відрізняються залежно від середовища.
- **Організація файлів:** Шаблони часто організовані за типом ресурсу, щоб визначення були модульними та читабельними, з окремими файлами для розгортань, служб, вхідного доступу та інших ресурсів.

Шаблонні директиви

Директиви шаблонів у діаграмах Helm — це команди, написані в синтаксисі шаблонів Go, які дозволяють генерувати динамічний контент у маніфестах Kubernetes. Вони дозволяють налаштовувати, використовувати умовну логіку та шаблони повторного використання в шаблонах діаграм. Ключові директиви включають:

- **Вставка змінних (`{{ .Values.key }}`):** Ця директива отримує значення з `Values.yaml`, замінюючи заповнювачі в шаблонах певними конфігураціями під час рендерингу. Наприклад, `{{ .Values.replicaCount }}` динамічно вставлятиме значення кількості реплік з `Values.yaml`.
- **Умовні оператори (`if/else/with`):** Helm дозволяє використовувати умовні оператори в шаблонах для ввімкнення або вимкнення ресурсів чи конфігурацій. Наприклад, ви можете використовувати `{{ if .Values.enableIngress }}` для додавання ресурсу вхідного типу, лише якщо це вказано у `Values.yaml`.
- **Цикли (діапазон):** Директива `range` виконує ітерацію по списках або словниках, що корисно для створення кількох ресурсів на основі списку значень. Це корисно для визначення кількох кінцевих точок служби або інших повторюваних елементів.
- **Функції:** Helm містить низку вбудованих функцій (наприклад, `upper`, `default`) для маніпулювання текстом, застосування значень за замовчуванням та виконання інших змін у даних.

Створення простої діаграми

Створення простої діаграми Helm включає кілька кроків, які призводять до базового пакета, готового до розгортання на Kubernetes. Ось спрощений огляд:

1. **Ініціалізація діаграми:** Використовуйте `helm create <ім'я-діаграми>` для створення шаблонної структури діаграми зі загальними каталогами та файлами (такими як `Chart.yaml`, `Values.yaml` та `templates/`). Ця структура надає вам шаблон для створення вашої програми.
2. **Визначення метаданих:** оновіть `Chart.yaml`, додавши назву, версію та опис діаграми, щоб точно описати розгортаний застосунок.
3. **Налаштування `Values.yaml`:** Встановіть значення за замовчуванням у `Values.yaml` для таких параметрів, як кількість реплік, назва образу та будь-які інші налаштовувані параметри. Ці значення слугуватимуть конфігурацією за замовчуванням.
4. **Редагування шаблонів:** Налаштуйте файли в `templates/`, щоб визначити ресурси Kubernetes (наприклад, `Deployment`, `Service`), використовуючи заповнювачі для значень, визначених у `Values.yaml`.
5. **Розгортання діаграми:** Використовуйте `helm install <release-name> <chart-name>` для розгортання діаграми, яка відобразить шаблони зі значеннями та застосуватиме їх до вашого кластера Kubernetes.

Тестування вашої діаграми

- **Лінтинг за допомогою helm lint:** Ця команда перевіряє наявність синтаксичних помилок, структурних проблем та поширених помилок у діаграмі. Це початкова перевірка для виявлення основних помилок перед розгортанням.
- **Пробний запуск з helm install --dry-run:** Запуск сухої інсталяції відтворює шаблони та виводить згенеровані маніфести Kubernetes без їх фактичного розгортання. Це корисно для перевірки синтаксису та значень шаблону.
- **Модульні тести за допомогою helm unittest:** Ви можете створювати тестові файли для перевірки певних умов шаблону. Такі інструменти, як helm unittest (плагін Helm), дозволяють перевірити, чи шаблони поведуться належним чином за різних конфігурацій.
- **Інсталяція в тестовому середовищі:** Розгортання діаграми в проміжному або тестовому середовищі – це практичний спосіб перевірити, чи всі ресурси налаштовані належним чином, а залежності та конфігурації працюють у режимі реального часу.

Найкращі практики для структури діаграми

- **Зберігайте модульність шаблонів:** упорядкуйте ресурси в окремі шаблони (наприклад, один для розгортання, один для обслуговування). Така модульність покращує читабельність та спрощує керування окремими ресурсами.
- **Використовуйте змістовні значення за замовчуванням у `Values.yaml`:** надайте конфіденційні значення за замовчуванням для часто налаштовуваних параметрів, щоб забезпечити використання діаграм "з коробки" з мінімальним налаштуванням.
- **Використовуйте умовну логіку:** використовуйте умовні оператори для додавання додаткових ресурсів лише за потреби (наприклад, `ingress.enabled`). Це забезпечує гнучкість діаграми для різних середовищ та потреб користувачів.
- **Параметри конфігурації документа:** Додайте файл `README.md`, який чітко пояснює призначення діаграми, як її використовувати та детально описує параметри, що налаштовуються, у файлі `Values.yaml`.
- **Ізоляція простору імен:** розробка діаграм для роботи в межах певного простору імен, щоб запобігти конфліктам з іншими програмами та покращити організацію кластера.



Розгортання за допомогою Helm Charts

Розгортання програм за допомогою Helm

- **Встановлення діаграми:** Використовуйте `helm install <release-name> <chart-name>` для розгортання діаграми. Helm відрендерить шаблони зі значеннями з `Values.yaml` та застосує ресурси Kubernetes до кластера. `<release-name>` однозначно ідентифікує цей екземпляр розгортання.
- **Налаштування користувацьких значень:** Під час розгортання ви можете перевизначити налаштування `Values.yaml`, передаючи користувацькі значення через окремий файл (`-f custom-values.yaml`) або безпосередньо за допомогою прапорця `--set`. Це дозволяє налаштовувати параметри відповідно до середовища.
- **Моніторинг розгортання:** Після розгортання ви можете контролювати стан програми за допомогою команд `kubectl` для перевірки подів, служб та інших ресурсів, створених діаграмою.
- **Оновлення та відкат:** Helm дозволяє безперешкодне оновлення до нових версій діаграми (оновлення `helm`) та легке відкатування (відкат `helm`) у разі виникнення проблем, що спрощує управління життєвим циклом програми.

Зняття Helm

- **Найменування випуску:** випуск ідентифікується назвою, вказаною під час розгортання (`helm install <releasename> <chart-name>`). Ця назва однозначно відрізняє його від інших розгортань тієї ж діаграми.
- **Версіонування та історія:** Helm веде історію версій для кожного випуску, записуючи кожне оновлення, відкат та застосовані зміни. Це дозволяє легко відстежувати зміни розгортання з часом.
- **Оновлення та відкати:** Релізи можна оновити до нових версій діаграм за допомогою оновлення `helm` та відкотити до попередніх версій за допомогою відкату `helm`, що пропонує контрольовані оновлення програм та варіанти відновлення.
- **Керування релізами:** такі команди, як `helm list` (для перегляду активних релізів) та `helm status <releasename>` (для отримання детальної інформації про певний реліз), допомагають керувати релізами в кластері Kubernetes.

Налаштування Values.yaml

Налаштування Values.yaml є ключовим для адаптації діаграми Helm до різних середовищ або конкретних вимог без зміни основних шаблонів діаграми. Ця гнучкість дозволяє користувачам змінювати такі параметри, як обмеження ресурсів, версії образів та кількість реплік, щоб відповідати конкретним потребам розгортання. Основні методи налаштування включають:

1. **Безпосереднє редагування значень:** Ви можете безпосередньо змінити Values.yaml, щоб встановити нові значення за замовчуванням для параметрів, які застосовуватимуться щоразу, коли розгортатиметься діаграма.
2. **Використання файлів користувацьких значень:** Замість редагування Values.yaml створіть окремі файли значень (наприклад, dev-values.yaml або prod-values.yaml) для підтримки конфігурацій, специфічних для середовища. Ці файли потім можна застосувати за допомогою прапорця -f (helm install -f prod-values.yaml <назва випуску> <назва-діаграми>).
3. **Перевизначення командного рядка:** Прапорець --set дозволяє тимчасово перевизначати значення з командного рядка, такі як --set replicaCount=3, що корисно для швидких або одноразових налаштувань.

Оновлення версії Helm

Оновлення випуску Helm дозволяє застосовувати зміни до розгорнутої програми без повного повторного розгортання. Цей процес включає зміну значень конфігурації або оновлення до нової версії діаграми.

Ключові кроки оновлення релізу включають:

1. **Зміна конфігурації:** оновіть значення у файлі Values.yaml або у файлі користувацьких значень, або скористайтесь параметром `--set` для тимчасового налаштування параметрів.
2. **Команда оновлення:** Використовуйте `helm upgrade <release-name> <chart-name>`, щоб застосувати нову конфігурацію або оновлену версію діаграми. Helm генеруватиме нові ресурси Kubernetes або оновлюватиме існуючі на основі змінених значень.
3. **Контроль версій:** Helm автоматично відстежує історію версій кожного випуску. Кожне оновлення створює нову ревізію, яку можна переглянути за допомогою історії `helm <release-name>`.
4. **Варіант відкату:** Якщо після оновлення виникають проблеми, `helm rollback <назва випуску> <редакція>` дозволяє легко повернути програму до попередньої версії.

Механізм відкату

Механізм відкату в Helm забезпечує важливу систему безпеки, дозволяючи командам швидко повертати програму до попереднього, стабільного стану, якщо оновлення або розгортання створює проблеми. Під час оновлення випуску Helm, Helm автоматично записує кожну версію розгортання, створюючи детальну історію, до якої можна буде звернутися пізніше. Якщо щось піде не так із поточним розгортанням, проста команда відкату відновить попередню версію, що дозволить відновити систему з мінімальними збоями. Ця можливість керування версіями та відкату є безцінною у виробничому середовищі, оскільки забезпечує швидке відновлення після помилок, зменшує час простою та захищає від потенційних неправильних конфігурацій або непередбачених проблем. Механізм відкату Helm — це потужний інструмент для забезпечення стабільності та надійності застосунків у Kubernetes.

Простори імен Helm та Kubernetes

Helm плавно інтегрується з просторами імен Kubernetes, дозволяючи розгортати додатки в певних, ізольованих середовищах кластера. Кожен випуск Helm може бути обмежений певним простором імен, що допомагає керувати ресурсами та організовувати їх, особливо в кластерах з кількома клієнтами або середовищах з кількома програмами. Простори імен підвищують безпеку, ізолюючи ресурси та запобігаючи конфліктам між різними випусками. У Helm ви можете вказати простір імен під час встановлення, гарантуючи, що всі ресурси для певного випуску розгортаються та керуються в межах цього призначеного простору. Ця функціональність не лише покращує організацію ресурсів, але й забезпечує більш детальний контроль над доступом та квотами на ресурси, що відповідає найкращим практикам Kubernetes щодо сегментації середовища та ізоляції ресурсів.

Робота з кількома релізами

У Helm робота з кількома релізами дозволяє розгортати кілька екземплярів однієї й тієї ж програми, кожен з яких має різні конфігурації або призначення, такі як окремі розгортання для розробки, проміжного тестування та виробництва. Кожен реліз Helm має унікальну назву, що дозволяє кільком версіям діаграми співіснувати в одному кластері або навіть в одному просторі імен, залежно від потреб конфігурації. Призначаючи унікальні назви та власні значення для кожного випуску, ви можете керувати ізольованими екземплярами та контролювати їхні конфігурації незалежно. Ця можливість особливо цінна для тестування та розробки, оскільки дозволяє розгортати ідентичні програми з різними налаштуваннями, а також підтримує швидку ітерацію та порівняння. Керування релізами Helm дозволяє кластерам Kubernetes підтримувати складні розгортання з кількома екземплярами, спрощуючи процес керування кількома версіями та конфігураціями в одному середовищі.

Перегляд розгорнутих релізів

Helm надає кілька команд для перегляду та керування розгорнутими релізами в кластері Kubernetes. Команда `helm list` надає огляд усіх активних релізів, показуючи назву кожного релізу, простір імен, стан та версію діаграми. Цей звіт дозволяє швидко оцінити, які програми наразі розгорнуті, та їхні відповідні конфігурації. Для отримання детальнішої інформації про певний випуск команда `helm status <release-name>` показує такі деталі, як ресурси Kubernetes, стан розгортання та історію останніх дій. Використовуючи ці команди, Helm забезпечує чіткий огляд усіх розгорнутих екземплярів, допомагаючи командам контролювати стан програм, виявляти будь-які потенційні проблеми та ефективно керувати ресурсами в усьому кластері.

Виправлення неполадок розгортання Helm

Виправлення неполадок у розгортаннях Helm передбачає поєднання команд Helm та інструментів Kubernetes для виявлення та вирішення проблем у розгортаннях застосунків. Якщо розгортання не вдається, початок зі значення `helm status <release-name>` може надати початковий огляд стану релізу, включаючи повідомлення про помилки або ресурси, які не вдалося розгорнути. Виконання команди `helm get manifest <release-name>` дозволяє переглянути фактичні маніфести Kubernetes, згенеровані з шаблонів, що може виявити помилки шаблонів або неправильно налаштовані значення. Крім того, команди `kubectl`, такі як `kubectl describe pod` або `kubectl logs`, пропонують глибше розуміння стану та журналів певних ресурсів, допомагаючи виявляти першопричини, такі як помилки вилучення зображень, проблеми з дозволами або обмеження ресурсів. Використання цих інструментів Helm та Kubernetes разом сприяє ефективному налагодженню, допомагаючи вирішувати проблеми з розгортанням та відновлювати стабільність програми.

Життєвий цикл ресурсу з Helm

Helm відіграє значну роль в управлінні життєвим циклом ресурсів Kubernetes, оптимізуючи процес створення, оновлення та видалення ресурсів за потреби. Коли діаграму Helm встановлено, Helm генерує та розгортає ресурси Kubernetes, ініціалізуючи застосунок та відстежуючи його як «реліз». Протягом життєвого циклу програми Helm може застосовувати оновлення шляхом оновлення, коригування ресурсів на основі оновлених конфігурацій або нових версій діаграм. За потреби Helm також може повернутися до попередніх версій, що дозволяє безпечно експериментувати та керувати версіями. Зрештою, коли реліз більше не потрібен, Helm може повністю видалити всі пов'язані ресурси за допомогою команди `helm uninstall`, видаливши як самі ресурси, так і дані відстеження релізу. Таке управління життєвим циклом забезпечує ефективний та повторюваний спосіб керування програмами, забезпечуючи плавний перехід від розгортання до оновлення, відкату та видалення в середовищі Kubernetes.



Розширені характеристики Helm

Гачки Helm

Перехоплювачі Helm дозволяють запускати користувацькі дії в певні моменти життєвого циклу релізу, забезпечуючи кращий контроль над складними розгортаннями. Хуки можна додавати до ресурсів у каталозі templates/ та запускати на заздалегідь визначених етапах, таких як до або після встановлення, оновлення чи видалення. Наприклад, хук з позначкою pre-install може виконати завдання для налаштування необхідних передумов перед розгортанням основної програми, тоді як хук post-delete може виконувати завдання очищення після видалення релізу. Ці перехоплювачі безцінні для оркестрації складних робочих процесів, таких як міграція баз даних або очищення ресурсів. Підтримуючи такі дії, керовані подіями, перехоплювачі Helm надають вам можливість керувати залежностями розгортання та готувати середовища, забезпечуючи безперебійний та надійний процес розгортання.

Тестування Helm

Helm-тести надають метод перевірки того, чи розгорнута програма функціонує належним чином у кластері Kubernetes. Ці тести визначені як завдання Kubernetes у діаграмі та запускаються командою `helm test <release-name>` після розгортання. Тест Helm може включати перевірку доступності бази даних, забезпечення правильної роботи служби або виконання перевірок справності ключових компонентів. Якщо тести пройдуть успішно, вони нададуть негайний зворотний зв'язок про те, що розгортання працює; якщо вони зазнають невдачі, вони висвітлюють проблеми, які потребують уваги. Тести Helm покращують процес розгортання, пропонуючи вбудований механізм для перевірки після розгортання, що сприяє підвищенню надійності та ранньому виявленню проблем у середовищах Kubernetes.

Піддіаграми та залежності

Піддіаграми та залежності в Helm дозволяють модульне керування діаграмами, що дозволяє включати та керувати пов'язаними сервісами як частиною основної діаграми. Піддіаграма — це, по суті, діаграма Helm, включена як залежність в іншу діаграму, зазвичай зазначену у файлі `Chart.yaml` у розділі залежностей. Таке налаштування корисне під час розгортання програм з кількома компонентами, таких як веб-програма, яка потребує окремої бази даних або служби кешування. Кожна піддіаграма працює незалежно в межах головної діаграми, що дозволяє її вмикати, налаштовувати або вимикати за потреби, не впливаючи на інші частини розгортання. Організовуючи програми за допомогою піддіаграм та залежностей, Helm сприяє створенню більш модульних, багаторазових та зручних у обслуговуванні конфігурацій, що спрощує керування складними багатосервісними розгортаннями в Kubernetes.

Керування файлами значень для різних середовищ

Helm спрощує керування конфігураціями для різних середовищ, таких як розробка, підготовка та виробництво, завдяки підтримці кількох файлів значень. Ви можете створити окремі файли значень (наприклад, `devvalues.yaml`, `prod-values.yaml`) для кожного середовища, кожен з яких містить конфігурації, специфічні для середовища, такі як кількість реплік, обмеження ресурсів або теги зображень. Під час розгортання ці файли можна вказати за допомогою прапорця `-f` (`helm install -f prod-values.yaml <назва випуску> <назва-діаграми>`), що дозволить Helm застосувати правильну конфігурацію для цього середовища. Такий підхід забезпечує узгодженість між розгортаннями, водночас дозволяючи контрольовані варіації, що спрощує адаптацію однієї діаграми Helm до кількох середовищ. Керуючи файлами значень окремо, ви упорядковуєте конфігурації та забезпечуєте безперебійний, масштабований робочий процес для розгортання програм на різних етапах життєвого циклу розробки.

Розширене шаблонування за допомогою Helm

Розширене створення шаблонів у Helm використовує потужний механізм шаблонів Go для динамічного налаштування ресурсів Kubernetes зі складною логікою та гнучкістю. Це включає функції, умовні оператори, цикли та вкладені шаблони, що дозволяють створювати складні конфігурації. Наприклад, ви можете використовувати функції для маніпулювання рядками або створення значень за замовчуванням, умовні оператори для включення або виключення ресурсів на основі певних параметрів та цикли для створення кількох екземплярів ресурсу зі списку. Крім того, Helm підтримує «іменовані шаблони», які дозволяють визначати фрагменти коду, що можна використовувати повторно, на які можна посылатися в різних шаблонах, що покращує зручність обслуговування та зменшує надмірність. Розширені шаблони особливо корисні в сценаріях, що вимагають складних конфігурацій або умовної логіки, що дозволяє діаграмам Helm легко адаптуватися до різноманітних та складних потреб розгортання.

Глобальні цінності в Helm

Глобальні значення в Helm надають спосіб визначення параметрів, доступних на всіх діаграмах та піддіаграмах у випуску Helm. На відміну від звичайних значень, які обмежені певною діаграмою, глобальні значення зберігаються під глобальним ключем у `Values.yaml` і на них можна посилалися з будь-якого шаблону на основній діаграмі або її залежностях. Наприклад, визначення глобального `database.url` у розділі `global` дозволяє як головній програмі, так і будь-яким залежним піддіаграмам використовувати один рядок підключення до бази даних. Такий підхід сприяє узгодженості та зменшує дублювання, особливо коли кілька компонентів покладаються на спільні конфігурації. Використовуючи глобальні значення, ви можете підтримувати єдину структуру конфігурації в багатосервісних застосунках, оптимізуючи управління та забезпечуючи узгодженість у всіх розгортаннях Helm.

Найкращі практики безпеки з Helm

Забезпечення безпеки розгортань Helm є критично важливим, оскільки діаграми Helm часто керують конфіденційними конфігураціями та доступом у кластерах Kubernetes. Ключові найкращі практики включають встановлення відповідних елементів керування доступом шляхом обмеження дозволів для користувачів Helm та забезпечення того, щоб діаграми не розкривали конфіденційні дані. Завжди зберігайте конфіденційні значення, такі як ключі API або паролі, в секретах Kubernetes, а не безпосередньо в `Values.yaml`, щоб запобігти розкриттю. Крім того, перед розгортанням важливо перевірити та переглянути діаграми із зовнішніх джерел, оскільки діаграми сторонніх розробників можуть містити вразливості. Видалення Tiller у Helm 3 підвищує безпеку, усуваючи необхідність у дозволах для всього кластера, що відповідає принципу керування доступом на основі ролей (RBAC) у Kubernetes. Дотримання цих практик допомагає мінімізувати ризики та забезпечити безпеку та контрольованість розгортання Helm.

Створення приватних репозиторіїв Helm

Створення приватного репозиторію Helm корисне для організацій, які бажають безпечно розміщувати та розповсюджувати внутрішні діаграми. Щоб налаштувати приватний репозиторій, ви можете використовувати такі рішення для зберігання даних, як Amazon S3, Google Cloud Storage або локальний сервер з доступом HTTP або HTTPS, у поєднанні з такими інструментами, як ChartMuseum або Harbor для розміщення діаграм. Діаграми можна упаковати за допомогою пакета helm, а потім завантажити до репозиторію, де вони індексуються та стають доступними для внутрішнього використання. Такі засоби контролю доступу, як автентифікація та авторизація, можна застосовувати для обмеження доступу лише авторизованими користувачами. Приватні репозиторії дозволяють командам безпечно керувати та розповсюджувати діаграми в організації, гарантуючи конфіденційність конфіденційних конфігурацій та власних програм, водночас використовуючи можливості управління пакетами Helm.

Керування сховищами діаграм

Керування репозиторіями діаграм у Helm дозволяє користувачам зберігати, упорядковувати та отримувати доступ до колекцій діаграм з різних джерел, спрощуючи розповсюдження та розгортання програм. Репозиторій діаграм може бути публічним, як-от репозиторії Helm за замовчуванням, або приватним внутрішнім репозиторієм, призначеним для використання в організації. Helm використовує команди `helm repo` для керування репозиторіями: `helm repo add` додає новий репозиторій, `helm repo update` синхронізує локальну інформацію з останніми діаграмами з кожного репозиторію, а `helm repo list` відображає доступні на даний момент репозиторії. Для безпечних середовищ керування приватними репозиторіями включає налаштування контролю доступу та HTTPS-з'єднань для забезпечення цілісності діаграм та обмеження несанкціонованого доступу. Ефективно організовуючи та керуючи сховищами діаграм, команди можуть оптимізувати доступ до діаграм, підтримувати узгодженість і гарантувати, що кожне розгортання має доступ до останніх, затверджених версій діаграм.



Практичне управління картою Helm

Автоматизація розгортання Helm за допомогою CI/CD

Автоматизація розгортання Helm за допомогою CI/CD безперешкодно інтегрує Helm у конвеєри безперервної інтеграції та безперервного розгортання, забезпечуючи швидші, надійніші та повторювані випуски додатків. У робочому процесі CI/CD діаграми Helm можуть розгортатися автоматично як частина конвеєра, ініційовані змінами в репозиторії коду або файлах конфігурації. Інструменти CI/CD, такі як Jenkins, GitLab CI або GitHub Actions, можуть використовувати команди Helm для встановлення, оновлення або відкату діаграм під час створення та тестування нових версій програм. Ця автоматизація гарантує, що кожне розгортання відповідає перевіреним конфігураціям, зменшуючи людські помилки та ручні зусилля. Крім того, завдяки включенню функції відкату Helm, конвеєри CI/CD можуть реалізовувати автоматичний відкат у разі збою розгортання, підвищуючи стабільність та стійкість виробничих систем.

Використання Helm з GitOps

Helm добре інтегрується з практиками GitOps, де вся інфраструктура та конфігурація застосунків зберігаються в репозиторіях Git, слугуючи єдиним джерелом достовірної інформації. Завдяки Helm у робочому процесі GitOps, діаграми та значення керуються в репозиторіях з контролем версій, а зміни в цих файлах автоматично запускають розгортання. Такі інструменти, як ArgoCD або Flux, можуть відстежувати зміни в репозиторіях та розгортати оновлені Helm-діаграми в Kubernetes, гарантуючи, що фактичний стан у кластері відповідає заявленому стану в Git. Такий підхід забезпечує декларативний, відстежуваний та аудитований процес розгортання, оскільки кожна зміна в інфраструктурі або конфігурації програми проходить через Git, що спрощує керування та відстеження модифікацій. Використання Helm з GitOps забезпечує узгодженість та надійність, узгоджуючи інфраструктуру як код з розгортанням додатків повністю автоматизованим способом.

Масштабування розгортань за допомогою Helm

Масштабування розгортань за допомогою Helm спрощує процес налаштування ресурсів додатків залежно від потреб, використовуючи вбудовані можливості масштабування Kubernetes. У `Values.yaml` можна вказати параметри, такі як `replicaCount`, щоб встановити початкову кількість реплік pod для застосунку. Шаблони Helm дозволяють легко налаштовувати ці значення для різних середовищ або динамічно оновлювати їх у рамках оновлення розгортання. Крім того, Helm може працювати з горизонтальним автомасштабуванням подів (HPA) від Kubernetes, додаючи конфігурації для запитів на ресурси та обмежень до діаграми. Ця інтеграція дозволяє програмам автоматично масштабуватися залежно від використання процесора або пам'яті, забезпечуючи ефективну адаптацію до вимог робочого навантаження. Гнучкість Helm в управлінні конфігураціями масштабування спрощує процес налаштування рівнів ресурсів, що полегшує підтримку продуктивності та стійкості програм за різних умов навантаження.

Версіонування в Helm Charts

Версіонування в діаграмах Helm відіграє життєво важливу роль у підтримці узгодженості, управлінні оновленнями та забезпеченні можливості відкату в різних розгортаннях програм. Кожна діаграма має власне поле версії в `Chart.yaml`, яке вказує версію самої діаграми, та поле `appVersion`, яке вказує на версію розгортаної програми. Ці поля дозволяють точно відстежувати зміни як у конфігураціях діаграми, так і в застосунку. Під час розгортання або оновлення випуску, зазначення конкретної версії діаграми гарантує застосування правильної, перевіреної конфігурації. Helm також відстежує версії випусків для кожного екземпляра розгортання, дозволяючи користувачам за потреби повернутися до певного випуску. Версіонування в Helm забезпечує чітку історію та контроль над конфігураціями програм, підтримуючи надійне керування розгортанням та легке усунення несправностей у складних середовищах Kubernetes.

Використання секретів Helm

Helm Secrets забезпечує безпечне керування конфіденційною інформацією, такою як ключі API, паролі та сертифікати, в межах діаграм Helm. За замовчуванням конфіденційні дані не повинні зберігатися у відкритому тексті в `Values.yaml`, оскільки це може розкрити секрети будь-кому, хто має доступ до діаграми. Helm Secrets дозволяє шифрувати ці значення, зазвичай використовуючи такі інструменти, як `sops` (Securely and Openly Protecting Secrets), для шифрування конфіденційної інформації у файлах значень. Ці зашифровані файли можна безпечно передати до системи контролю версій та розшифрувати лише за потреби під час розгортання. Такий підхід гарантує захист конфіденційних даних протягом усього процесу розгортання, підвищуючи безпеку та відповідність вимогам у середовищах, де конфіденційність та конфіденційність є критично важливими. Helm Secrets забезпечує рівень безпеки, який дозволяє командам безпечно керувати конфігураціями, не ставлячи під загрозу конфіденційну інформацію.

Багатокластерні розгортання

Helm підтримує багатокластерні розгортання, що дозволяє організаціям керувати програмами в кількох кластерах Kubernetes з однієї діаграми. Багатокластерні налаштування поширені в архітектурах високої доступності, гібридних хмарних середовищах або глобальних програмах, які потребують розподілених ресурсів по регіонах. За допомогою Helm ви можете налаштувати розгортання для цільової аудиторії, налаштувавши контексти Kubernetes для кожного кластера та вказавши контекст під час розгортання (`--kube-context`). Кожен кластер може використовувати спеціалізований файл `Values.yaml` для відображення налаштувань, специфічних для середовища, таких як розподіл ресурсів, регіони або репліки, що забезпечує узгодженість між кластерами та адаптацію до локальних вимог. Керування багатокластерними розгортаннями за допомогою Helm забезпечує гнучкість, стійкість та ефективний розподіл ресурсів, підтримуючи масштабовані, глобально розподілені програми з централізованим керуванням конфігурацією.

Перевірка та лінтування діаграм

Перевірка та лінтування діаграм є важливими кроками для забезпечення якості та надійності діаграм Helm перед розгортанням. Helm надає команду `helm lint` для аналізу діаграм на наявність синтаксичних помилок, структурних проблем та порушень найкращих практик. Ця початкова перевірка допомагає виявити поширені помилки, такі як відсутні поля або неправильно налаштовані шаблони, покращуючи узгодженість діаграм та зменшуючи ризик проблем із розгортанням. Окрім лінтування, перевірка діаграм включає перевірку шаблонів за допомогою «пробного запуску» (`helm install --dry-run`) для відображення маніфестів без їх фактичного розгортання, що дозволяє переглянути, як будуть генеруватися ресурси Kubernetes. Ці кроки життєво важливі для контролю якості, допомагаючи командам створювати стабільні та добре перевірені діаграми, які надійно працюють у виробничому середовищі.

Екосистема плагінів Helm

Екосистема плагінів Helm розширює основний функціонал Helm додатковими інструментами та можливостями для покращення процесів керування діаграмами та розгортання. Плагіни – це доповнення, які можна встановлювати для автоматизації завдань, підвищення безпеки та інтеграції з іншими інструментами DevOps. Наприклад, плагіни, такі як `helmsecrets`, забезпечують безпечну обробку конфіденційних даних, тоді як `helm-diff` забезпечує візуальне порівняння між версіями діаграм, допомагаючи з контролем версій та прийняттям рішень щодо оновлення. Інші плагіни можуть оптимізувати тестування, керувати кількома кластерами Kubernetes або покращувати робочі процеси CI/CD. Відкритий характер екосистеми плагінів Helm дозволяє командам налаштовувати Helm відповідно до своїх конкретних потреб, підвищуючи його гнучкість та функціональність для підтримки складних середовищ та робочих процесів Kubernetes.

Розширене управління релізами

Розширене керування релізами в Helm забезпечує детальний контроль над розгортаннями програм, що спрощує складні стратегії оновлення, відкату та управління версіями. Завдяки функції історії релізів Helm кожна дія оновлення або відкату зберігається як нова версія релізу, що дозволяє відстежувати зміни з часом. Ця історія дозволяє легко повернутися до попереднього стабільного стану, якщо виникають проблеми, забезпечуючи мінімальний час простою. Helm також дозволяє виконувати автоматичні відкати, що запускаються інструментами моніторингу або конвеєрами CI/CD у відповідь на невдалі розгортання, підвищуючи стійкість. Крім того, використовуючи попередньо налаштовані файли значень для різних версій, команди можуть керувати поетапними розгортаннями, такими як канарейкові розгортання або синьо-зелені стратегії, де спочатку оновлюються певні середовища для тестування. Розширене керування релізами в Helm забезпечує гнучкість, необхідну для ефективного керування складними, ітеративними оновленнями програм, підтримуючи стабільність у динамічних середовищах Kubernetes.

Стратегії резервного копіювання та відновлення за допомогою Helm

Впровадження стратегій резервного копіювання та відновлення за допомогою Helm є важливим для аварійного відновлення та підтримки безперервності стану програм у Kubernetes. Сам Helm не створює резервні копії даних, але може керувати резервними копіями конфігурації, експортуючи значення та метадані випуску. Команда `helm get values <release-name>` отримує значення конфігурації випуску, які можна зберігати як резервну копію для повторного розгортання. Крім того, ресурси Kubernetes, створені Helm, такі як Persistent Volume Claims (PVC), можуть потребувати окремого резервного копіювання за допомогою рішень рівня сховища або сторонніх інструментів. У разі збою або втрати даних, повторне розгортання діаграми Helm за допомогою збережених файлів конфігурації та відновлення даних з резервних копій PVC може відтворити попередній стан програми. Інтегруючи Helm з рішеннями для зберігання даних та керуючи резервними копіями конфігурації, команди можуть створювати надійні процеси відновлення, які забезпечують швидке відновлення та стійкість програм.

Питання та відповіді